Prompt Engineering — Assessment

Let's Discuss: Reflection Questions & Answers

Name:	Y RAVI TEJA
Register No:	99230040828
Email:	99230040828@klu.ac.in
Date:	September 5, 2026

---

1. What is the advantage of using {user_input} instead of hard-coding input?

Using a placeholder like `{user_input}` makes a prompt a reusable template instead of a one-off string. The same prompt structure, instructions, and formatting can then be applied to any new input simply by substituting a different value into the placeholder, rather than rewriting the whole prompt each time. This makes prompts easier to maintain, test, and scale across many users or many pieces of input data, and it separates the fixed 'instruction' part of the prompt from the variable 'content' part.

2. When would .format() be enough?

Python's `.format()` (or an f-string) is enough when the prompt template is simple: a fixed block of text with one or more straightforward variable substitutions and no conditional logic, loops, or repeated sections. For example, a template like `"Summarize the following text: {text}"` only needs a plain string substitution, so a lightweight tool like .format() is sufficient and avoids the overhead of a full templating engine.

3. When might Jinja2 be useful?

Jinja2 becomes useful once a prompt template needs more than simple substitution — for example, conditional blocks (include a section only if a variable is present), loops (insert a list of examples or documents dynamically), or nested/structured data. It lets prompt engineers build complex, dynamic prompts (such as few-shot prompts with a variable number of examples, or prompts that change instructions based on user type) in a clean, readable, and maintainable way that plain string formatting cannot easily handle.

4. Why should prompts be versioned?

Prompts directly control model behavior, so even small wording changes can change outputs in significant ways. Versioning prompts (e.g., keeping a history of prompt_v1, prompt_v2, etc.) allows teams to track what changed, why it changed, and what effect the change had. It makes it possible to roll back to a previous version if a new prompt performs worse, to compare performance across versions, and to maintain accountability and reproducibility in how the system behaves over time.

5. What is a prompt regression?

A prompt regression occurs when a change to a prompt — intended to fix or improve one thing — unintentionally makes the model's output worse on cases it previously handled well. In other words, the new version of the prompt "regresses" in quality on certain inputs even while it may improve

performance on others. Regressions are the reason prompt changes need to be tested against a broad set of cases before being deployed.

6. Why use the same evaluation set when comparing prompt versions?

Using the same fixed evaluation set ensures that any difference in results between two prompt versions is due to the change in the prompt itself, and not due to a difference in the test data. This gives a fair, controlled, apples-to-apples comparison, similar to keeping a single variable constant in a scientific experiment. Without a consistent evaluation set, it becomes difficult to tell whether a prompt version genuinely improved performance or simply appeared better because it was tested on easier examples.